

PROGETTO 7: RICOSTRUZIONE DI UN GRAFO TOPOLOGICO DA MISURAZIONI DELLA DIFFUSIONE

SOMMARIO. Progetto di calcolo scientifico avanzato: 3 punti per la soluzione del Problema 1, 4 punti per la soluzione dei problemi 1 e 2 (si veda sotto).

1. CONTESTO E FINE

1.1. Laplaciano, equazione del calore e separazione delle variabili. Il laplaciano è uno degli oggetti più ubiquitari della matematica: il suo uso varia dalle applicazioni fisiche, all'analisi degli spazi di funzioni, dall'analisi di processi stocastici alla teoria dei numeri, il Laplaciano è estensivamente utilizzato nella modellazione matematica nelle scienze applicate, fino allo studio dei grafi.

Per una funzione di n variabili reali, derivabile due volte con continuità, si pone

$$\Delta f(x) := \sum_{i=1}^n \frac{\partial^2}{\partial x_i^2} f(x),$$

dove $\partial/\partial x_i$ è la derivata parziale, ovvero la derivazione rispetto alla variabile x_i trattando come costanti le x_j (con j diverso da i). E' bene ricordare che questo modo di definire il Laplaciano è il più semplice e storicamente il primo, tuttavia nella matematica moderna si usano definizioni che si basano su richieste molto meno stringenti su f che l'essere derivabile due volte con continuità.

Dal punto di vista intuitivo, il valore di $\Delta f(\bar{x})$ misura quanto f differisce dalla sua media locale attorno a $\bar{x}$ ed è nullo precisamente quando, per ogni $r > 0$,

$$f(\bar{x}) = \frac{1}{|B(\bar{x}, r)|} \int_{B(\bar{x}, r)} f(x) dx,$$

dove $B(\bar{x}, r)$ è la palla n -dimensionale di raggio r centrata in $\bar{x}$ e $|B(\bar{x}, r)|$ la sua misura.

Una delle principali applicazioni del Laplaciano è nella modellazione dei processi diffusivi, dove una certa quantità si disperde nello spazio in modo isotropo in tutte le direzioni, con effetto immediato anche a distanza infinita, ma che decade rapidamente allontanandosi nello spazio. L'esempio principe di questo tipo di fenomeni è l'equazione del calore

$$\begin{cases} \frac{\partial}{\partial t} u(x, t) = \Delta u(x, t) & x \in \mathbb{R}, t > 0 \\ u(x, 0) = u^0(x) & x \in \mathbb{R} \end{cases},$$

dove $u(x, t)$ è la temperatura nel punto x al tempo t e u^0 è una data distribuzione di temperatura iniziale. Intuitivamente, in questa dinamica la quantità u valutata nel punto x varia nel tempo a seconda di quanto $u(x, \dots)$ differisce dalla sua media attorno a x : se u è maggiore della media allora diminuirà, mentre se è minore della media aumenterà.

Si noti che l'incognita in tale equazione è u , una funzione! Questo tipo di (importantissime!) equazioni sono dette *equazioni alle derivate parziali* (PDEs in inglese). In questo setting estremamente semplificato, esiste una soluzione (semi-)esplicita dell'equazione del calore, data da

$$u(x, t) = \int_{\mathbb{R}} \frac{1}{(4\pi t)^{n/2}} e^{-\frac{\|x-y\|_2^2}{4t}} u^0(y) dy.$$

La funzione integranda che viene moltiplicata per il dato iniziale prende il nome di *nucleo del calore*.

In situazioni geometriche più complesse, come ad esempio una regione limitata dello spazio o una superficie compatta, un simile approccio è comunque percorribile. L'idea chiave è una versione estremamente generalizzata del teorema spettrale! Consideriamo il problema agli autovalori

$$\begin{cases} \Delta u = \lambda u & x \in \Omega \\ \partial_n u = 0 & x \in \partial\Omega \end{cases},$$

dove Ω è un aperto con bordo $\partial\Omega$ e $\partial_n u(x)$ denota la derivata di u nella direzione normale uscente da Ω . Accade che tale problema ha soluzione per $\lambda \in \sigma(\Delta) \subset (-\infty, 0]$, dove $\sigma(\Delta)$ è un insieme *numerabile* di valori, detto spettro. In particolare esistono le *autofunzioni* $\phi_\lambda : \Omega \rightarrow \mathbb{R}$ (sono l'equivalente degli autovettori!) per cui $\Delta\phi_\lambda = \lambda\phi_\lambda$ e queste sono *ortonormali* rispetto al prodotto

$$(\phi, \psi) := \int_{\Omega} \phi(x)\psi(x)dx.$$

Useremo queste funzioni come base di uno spazio vettoriale (infinito dimensionale) dove costruire la soluzione.

Immaginiamo ora di scrivere la soluzione dell'equazione del calore su Ω come espansione sulle ϕ_λ . Avremmo, per ogni $t \geq 0$,

$$\begin{aligned} u(x, t) &= \sum_{\lambda \in \sigma(\Delta)} u_\lambda(t) \phi_\lambda(x) \\ \Delta u(x, t) &= \sum_{\lambda \in \sigma(\Delta)} u_\lambda(t) \Delta \phi_\lambda(x) = \sum_{\lambda \in \sigma(\Delta)} \lambda u_\lambda(t) \phi_\lambda(x) \\ \frac{\partial}{\partial t} u(x, t) &= \sum_{\lambda \in \sigma(\Delta)} \frac{\partial}{\partial t} u_\lambda(t) \phi_\lambda(x) \end{aligned}$$

da cui, facendo il prodotto scalare per ϕ_η con $\eta \in \sigma(\Delta)$ e sfruttando l'ortonormalità della base, otteniamo

$$\begin{aligned} (u(\cdot, t), \phi_\eta) &= \sum_{\lambda \in \sigma(\Delta)} u_\lambda(t) (\phi_\lambda, \phi_\eta) = u_\eta(t) \\ \left(\frac{\partial}{\partial t} u(x, t), \phi_\eta\right) &= \sum_{\lambda \in \sigma(\Delta)} \frac{\partial}{\partial t} (u_\lambda(t) (\phi_\lambda, \phi_\eta)) = \frac{\partial}{\partial t} u_\eta(t) \\ (\Delta u(x, t), \phi_\eta) &= \sum_{\lambda \in \sigma(\Delta)} \lambda u_\lambda(t) (\phi_\lambda, \phi_\eta) = \eta u_\eta(t). \end{aligned}$$

Dunque i coefficienti di u sulla base $\{\phi_\lambda\}_{\lambda \in \sigma(\Delta)}$ soddisfano

$$u_\lambda(0) = (u^0, \phi_\lambda), \quad \frac{d}{dt} u_\lambda(t) = \lambda u_\lambda(t).$$

Queste sono equazioni differenziali ordinarie del prim'ordine che hanno la soluzione

$$u_\lambda(t) = (u^0, \phi_\lambda) e^{\lambda t}.$$

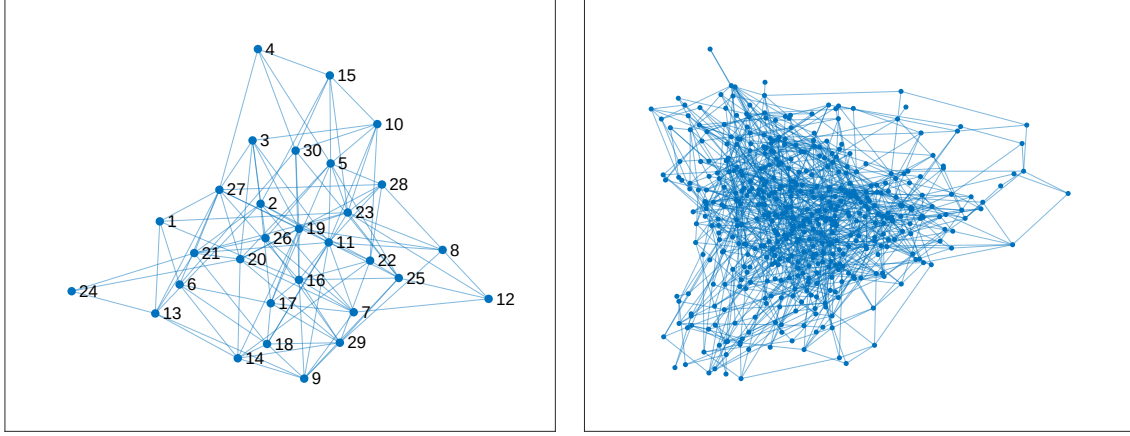
La costruzione che abbiamo seguito si chiama separazione delle variabili (spazio e tempo). Dunque abbiamo

$$u(x, t) = \sum_{\lambda \in \sigma(\Delta)} (u^0, \phi_\lambda) \phi_\lambda(x) e^{\lambda t} = (u^0(\cdot), K(\cdot, x, t)).$$

dove $K(y, x, t) = \sum_{\lambda \in \sigma(\Delta)} e^{\lambda t} \phi_\lambda(x) \phi_\lambda(y)$ è il nucleo del calore in questo setting.

Con il tempo si è capito che la conoscenza del nucleo del calore da' una profonda comprensione della geometria del dominio su cui l'equazione è stata risolta. Vediamo nella prossima sezione come questa idea si adatti perfettamente al contesto dei grafi.

FIGURA 1. Un esempio di grafo con qualche decina di nodi sulla sinistra e con 500 nodi sulla destra.



1.2. Diffusione su un grafo. Un grafo topologico è il dato di un insieme finito di *nod*i o *vertici* $V = \{v_1, \dots, v_N\}$ e di una relazione $\sim$ antiriflessiva simmetrica non transitiva su di essi, tale per cui vale la seguente proprietà: per ogni coppia $(i, j) \in \{1, \dots, N\}^2$ con $i \neq j$, esiste un sottoinsieme $\{k_1, k_2, \dots, k_M\}$ di $\{1, \dots, N\}$ per cui

$$v_i \sim v_{k_1} \sim v_{k_2} \cdots \sim v_{k_M} \sim v_j.$$

E' piuttosto comune e naturale (vedi Figura ??) pensare ad un grafo topologico come a dei punti nel piano eventualmente connessi da segmenti, in modo che si possa andare da ogni punto fino ad ogni altro usando i segmenti (infatti ci si riferisce a tale proprietà come connessione). Spesso infatti si definisce un grafo G assegnando la lista V dei vertici e quella degli edge e (i segmenti sopracitati), scrivendo $G = (V, e)$. Normalmente si rappresenta la relazione $\sim$ con una matrice $A \in M_{N \times N}(\{0, 1\})$, detta di adiacenza, dove

$$A(i, j) = \begin{cases} 1 & \text{se } v_i \sim v_j \\ 0 & \text{altrimenti} \end{cases}.$$

Ovviamente le proprietà di $\sim$ si possono ritrovare in A : sulla diagonale avremo tutti valori nulli (antiriflessività), A è simmetrica (simmetria), se $A(i, j) = 1$ e $A(j, k) = 1$, e non necessariamente avremo $A(i, k) = 1$ (non transitività). La connessione si traduce nel fatto che data una qualsiasi coppia di indici distinti i e j , possiamo passare dalla riga i alla riga o colonna j di A "camminando su gli uni della matrice e facendo solo passi verticali o orizzontali".

Un'altra quantità importante associata ad un grafo è il grado d di ogni nodo, ovvero il numero di nodi che è in relazione con esso. Possiamo allora definire la matrice Laplaciana del grafo come

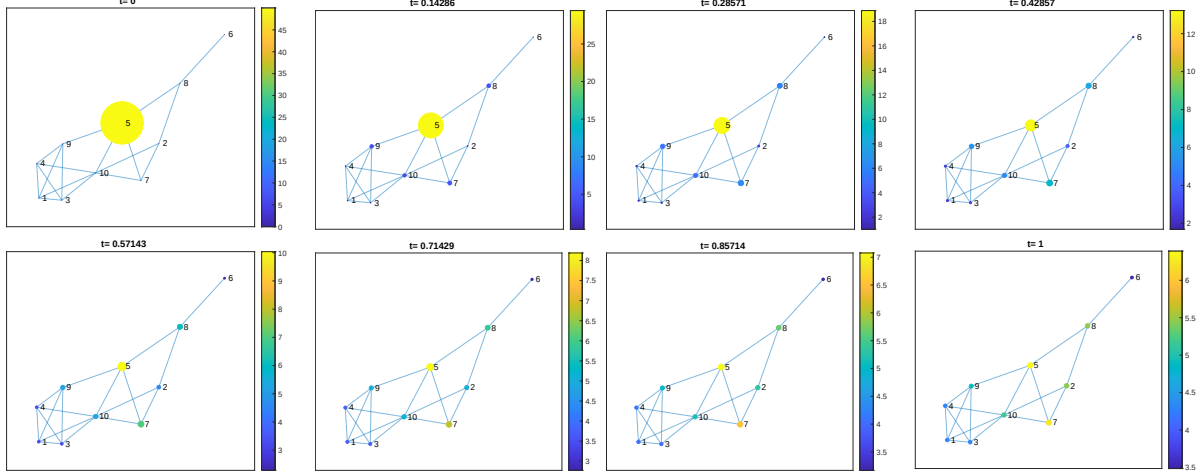
$$L = D - A, \quad D(i, j) = d_i \delta_{i,j}.$$

Questa matrice può essere pensata come applicazione lineare che ad una funzione u sul grafo, ovvero un vettore $(y_1, \dots, y_N)^t = (u(v_1), \dots, u(v_N))^t$ associa il vettore contenente i valori del Laplaciano di f **cambiati di segno**:

$$-\begin{pmatrix} \Delta u(v_1) \\ \vdots \\ \Delta u(v_N) \end{pmatrix} = L \begin{pmatrix} u(v_1) \\ \vdots \\ u(v_N) \end{pmatrix}.$$

Si noti che $Lu(v_i) = 0$ se e solo se $u(v_i)$ è pari alla media degli $u(v_j)$ con $i \sim j$.

FIGURA 2. Evoluzione della diffusione su un piccolo grafo partendo dal dato iniziale $u(v_i) = 50\delta_{i,5}$, otto istanti temporali equispaziati in $[0, 1]$.



L'equazione del calore sul grafo diventa dunque

$$(1) \quad \begin{cases} \frac{\partial}{\partial t} \mathbf{u}(t) = -L\mathbf{u}(t) & t > 0 \\ \mathbf{u}(0) = \mathbf{u}^0 \end{cases},$$

dove i grasseti indicano i vettori colonna ottenuti sovrapponendo (in colonna) le valutazioni su tutti i nodi del grafo.

Questa volta lo spazio di funzioni è necessariamente finito dimensionale (essendo spannato dalle funzioni che valgono 1 su un nodo e zero sugli altri) e le autofunzioni del Laplaciano altro non sono che gli autovettori di L . Se usiamo la decomposizione spettrale $L = P\Lambda P^t$ con Λ matrice diagonale con diagonale di reali non negativi e P matrice ortogonale, abbiamo che

$$\mathbf{u}(t) = \sum_{j=1}^N P(:,j)v_j(t),$$

e necessariamente i coefficienti di u sulla base di autovettori soddisfano

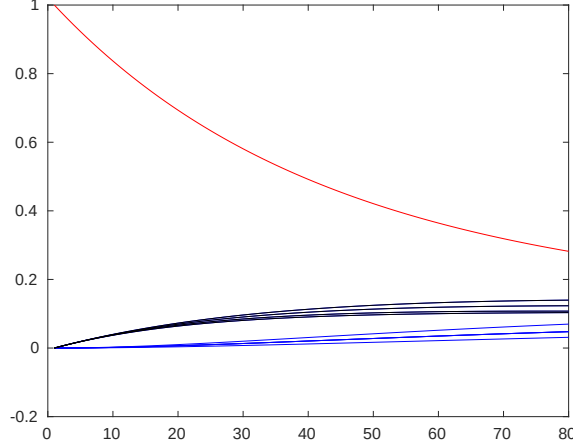
$$\begin{cases} \frac{d}{dt} v_j(t) = -\lambda_j v_j(t) & t > 0 \\ v_j(0) = P(:,j)^t \mathbf{u}^0 \end{cases} \Rightarrow v_j(t) = e^{-\lambda_j t} P(:,j)^t \mathbf{u}^0.$$

In altre parole abbiamo

$$\mathbf{u}(t) = \begin{pmatrix} u(v_1, t) \\ u(v_2, t) \\ \vdots \\ u(v_N, t) \end{pmatrix} = P \begin{bmatrix} e^{-\lambda_1 t} & 0 & \dots & \dots & 0 \\ 0 & e^{-\lambda_2 t} & 0 & \dots & 0 \\ \vdots & \vdots & \vdots & \vdots & \vdots \\ \vdots & \vdots & \vdots & \vdots & \vdots \\ 0 & \dots & \dots & \dots & e^{-\lambda_N t} \end{bmatrix} P^t \begin{pmatrix} u^0(v_1) \\ u^0(v_2) \\ \vdots \\ u^0(v_N) \end{pmatrix} = e^{-tL} \mathbf{u}^0.$$

1.3. Ricostruzione del grafo da "snapshots" dell'equazione del calore. Supponiamo ora di sapere che un dato fenomeno evolve su un grafo per diffusione (ovvero la quantità osservata $u(v_i, t)$ soddisfa l'equazione del calore sul grafo), ma di *non conoscere il grafo*. Possiamo ricostruire il grafo dalle misurazioni di $u(v_i, t)$ ($t > 0$ in un insieme finito di istanti)?

FIGURA 3. Evoluzione temporale del valore della soluzione su ciascun nodo: in rosso il nodo v_5 , in nero i nodi $v_j \sim v_5$, $j \neq 5$, e in blu tutti gli altri.



Pensiamo alla seguente euristica. Se il sistema partisse dalla soluzione che vale 1 in un nodo (diciamo i -esimo) e zero in tutti gli altri, come nell'esperimento riportato in figura ??, allora i nodi vicini a v_i avrebbero una veloce variazione dei valori di u nel tempo, mentre quelli distanti (nel senso del numero di edge necessari per raggiungerli) avrebbero una lenta variazione. Si noti che questo ragionamento, seppur puramente euristico, ha perfettamente senso

- per tempi sufficientemente brevi
- per distinguere i nodi a distanza di un edge dagli altri, non di più.

Questo è dovuto al fatto che la topologia di un grafo può essere estremamente complessa e strutture come i cicli possono influenzare pesantemente la diffusione.

Il primo problema che si vuole risolvere è dunque il seguente:

Problema 1 (Ricostruzione di un grafo da dati diffusivi sovrabbondanti). Data la lista degli m nodi $V = \{v_1, \dots, v_m\}$, una serie finita di istanti temporali $0 = t_0 < t_1 < \dots < t_n = T$ ed la misurazione di $M \geq m$ processi diffusivi $U\{j\}(i, k) := u^{(k)}(v_i, t_j) + \epsilon_{i,j,k}$, con $i = 1, \dots, m$, $j = 1, \dots, n$ e $k = 1, 2, \dots, M$, e dove $\epsilon_{i,j,k}$ è un errore di misurazione, trovare la matrice di adiacenza A del grafo.

Si noti che la notazione $U\{j\}(i, k)$ è consistente con quella che matlab utilizza per le celle. I dati disponibili in download sono di questo tipo, si veda `cell` nella documentation di matlab. Si noti anche la differente posizione degli indici... è un suggerimento per una programmazione più semplice ed efficiente.

L'idea di disporre di un numero di snapshots M maggiore o uguale al numero dei nodi del grafo è, nelle applicazioni reali, del tutto irrealizzabile.

Il problema che è più naturale è invece il seguente:

Problema 2 (Ricostruzione di un grafo da dati diffusivi lacunosi). Data la lista degli m nodi $V = \{v_1, \dots, v_m\}$, una serie finita di istanti temporali $0 = t_0 < t_1 < \dots < t_n = T$ ed la misurazione di $M < m$ processi diffusivi $u^{(k)}(v_i, t_j)$ con $i = 1, \dots, m$, $j = 1, \dots, n$ e $k = 1, 2, \dots, M$, trovare la matrice di adiacenza A del grafo.

2. SOLUZIONE DEL PROBLEMA ??

Il Problema ?? può essere facilmente attaccato con tecniche di approssimazione ai minimi quadrati. Possiamo procedere come segue:

- Per ogni nodo v_l , $i = l, \dots, m$, consideriamo il dato iniziale e_l (tale per cui $e_l(v_j) = \delta_{l,j}$) e cerchiamo di ricostruire la soluzione dell'equazione

$$\begin{cases} \frac{\partial}{\partial t} \mathbf{E}_l(t) = -L \mathbf{E}_l(t) & t > 0 \\ \mathbf{E}_l(0) = \mathbf{e}_l \end{cases}$$

- Consideriamo e risolviamo (metodo svd) il problema di approssimazione ai minimi quadrati di minima norma di $\mathbf{e}_l$ sulla spazio spannato dai vettori $U\{1\}(:, 1), \dots, U\{1\}(:, M)$, ottenendo

$$\mathbf{e}_l \approx \sum_{k=1}^M U\{1\}(:, k) c_k,$$

da cui segue immediatamente che

$$E_l(v_i, t_j) \approx \sum_{k=1}^M U\{j\}(v_i, k) c_k.$$

- Consideriamo le traiettorie $(E_l(v_i, t_1), \dots, E_l(v_i, t_{n'}))$, $i \neq l$, $j = 1, 2, \dots, n'$ con $n' \ll n$ ed approssimiamo la derivata temporale come $\frac{d}{dt} E_l(v_i, t_{j+1}) \approx (E_l(v_i, t_{j+1}) - E_l(v_i, t_j)) / (t_{j+1} - t_j)$. Utilizziamo l'algoritmo di clusterizzazione dati **kmeans** (disponibile nella *statistics and machine learning toolbox* di matlab¹, si veda la documentazione matlab per il suo utilizzo) per dividere le traiettorie in due cluster.
- Poniamo $A_{l,j} = A_{l,j} = 1$ per ogni j appartenente al cluster con derivata temporale più grande.

3. SOLUZIONE DEL PROBLEMA ??

Il secondo problema è più difficile perchè supponiamo di avere solo poche traiettorie a disposizione. Tuttavia possiamo fare un'osservazione utile: se $\mathbf{u}$ risolve l'equazione del calore con dato iniziale $\mathbf{u}(\cdot, 0)$, allora, per ogni $s > 0$, la funzione $\mathbf{u}(\cdot, s + \cdot)$ risolve la stessa equazione con dato iniziale $\mathbf{u}(\cdot, s)$.

Allora, invece che cercare di ricostruire la soluzione di

$$\begin{cases} \frac{\partial}{\partial t} \mathbf{E}_l(t) = -L \mathbf{E}_l(t) & t > 0 \\ \mathbf{E}_l(0) = \mathbf{e}_l \end{cases}$$

sullo spazio generato dai vettori $U\{1\}(:, 1), \dots, U\{1\}(:, M)$, possiamo cercare di farlo sullo spazio generato da

$$U\{1\}(:, 1), \dots, U\{1\}(:, M), U\{2\}(:, 1), \dots, U\{2\}(:, M), \dots, U\{n - n'\}(:, 1), \dots, U\{n - n'\}(:, M).$$

Notiamo che, se il rango della matrice di Vandermonde rettangolare $m \times (M \cdot (n - n'))$ è pieno, i minimi quadrati di minima norma sono ben posti (e risolvibili con svd) e la situazione in cui ci ritroviamo è la medesima che quella affrontata nel problema 1.

4. IMPLEMENTAZIONE E TEST DEL CODICE

La soluzione del Problema 1 (o 2) si può implementare in uno script **Solutore1.m** che prende i dati in input leggendo file di tipo ".mat" da disco fisso, si veda **load** nella documentazione matlab.

Prima di lanciare il codice prodotto sui dati forniti dal docente, il codice va testato. A tal fine, si crei uno script **buildsyntheticdata.m** dove

¹Si noti che se non si è installata la toolbox durante la prima installazione di matlab, si può aggiungere la toolbox velocemente utilizzando il pulsante "add-ons" di matlab.

- Si definisca un piccolo grafo ($m=10-20$ nodi) creando la matrice di adiacenza **A** e utilizzando la function **graph**. Si veda la documentazione a [questo link](#) per creare i plot di un grafo e controllare i parametri di rendering di vertici ed edges.
- Si costruisca la matrice Laplaciana del grafo (si usi **laplacian**) e se ne calcoli la diagonalizzazione $L = PAP'$ con la function **eig**.
- Sia **t=linspace(0,T,nt)** ($T \approx 0.1$, $nt \approx 50$), si crei una variabile **U** di tipo cella nella quale **U{1}=randn(m,m)** è una matrice $m \times m$ in cui la k -esima colonna $(u^{(j)}(v_1, 0), \dots, u^{(j)}(v_m, 0))^t$ rappresenta il dato iniziale random per il j -esimo processo di diffusione.
- Con un ciclo si riempiano i rimanenti **nt**-1 elementi di **U** usando il processo di diffusione:

$$U\{j\}(:,k) = \begin{pmatrix} u^{(k)}(v_1, j) \\ \vdots \\ u^{(k)}(v_m, j) \end{pmatrix} = P \begin{bmatrix} e^{-\lambda_1 t_k} & 0 & 0 & \dots \\ 0 & e^{-\lambda_2 t_k} & 0 & \dots \\ \vdots & \vdots & \vdots & \vdots \\ 0 & \dots & 0 & e^{-\lambda_m t_k} \end{bmatrix} P' \begin{pmatrix} u^{(k)}(v_1, 1) \\ \vdots \\ u^{(k)}(v_m, 1) \end{pmatrix}$$

- Si salvino su disco **A**, **t**, e **U** (vedi help del comando **save**).

In fine si testi il solutore (del problema 1) precedentemente implementato su questi dati sintetici controllando che la matrice di adiacenza calcolata corrisponda alla matrice salvata su disco.

Se il codice risulta corretto e funzionante sui dati sintetici si può risolvere il Problema 1 con i dati forniti dal docente.

In secondo luogo (**facoltativo**) si può modificare **Solutore1.m** per ottenere un file **Solutore2.m** in cui si risolve il problema 2. Si noti che la condizione di rango pieno va verificata (creare un ciclo while su n').